

Exercise 5: Real-Time Sync with SignalR

Context: Relying on browser polling loops via `setInterval` creates significant network overhead, delays critical layout updates, and fails to scale under hundreds of concurrent users. SignalR eliminates this constraint by maintaining a persistent WebSocket transport connection directly between client browsers and your backend server. Whenever an architecture state changes (such as an enrollment getting approved or a grade posted), the server immediately broadcasts the payload down to active clients in real time.

Step 1: Extend the Backend Hub Client Interface

Open your .NET core assembly file `TmsApi.Application/Hubs/ITmsHubClient.cs` and append the missing broadcast contract signature:

```
// File: TmsApi.Application/Hubs/ITmsHubClient.cs
public interface ITmsHubClient
{
    // New: broadcast enrollment status changes to all connected clients
    Task ReceiveEnrollmentStatusUpdated(string enrollmentId, string
status);
}
```

Note: Because your backend `TmsHub` extends the strongly-typed `Hub<ITmsHubClient>` interface, this method becomes immediately available across all target selection parameters with full compiler checking.

[!NOTE]

Step 2: Broadcast from the Enrollment Approval Endpoint

Open your .NET controller class file

`TmsApi.Api/Controllers/V2/EnrollmentsController.cs`. Inject the system hub context and dispatch the live change notification immediately after a database write transaction resolves successfully:

```
// File: TmsApi.Api/Controllers/V2/EnrollmentsController.cs
public class EnrollmentsController(
    /* your existing dependencies */
    IHubContext<TmsHub, ITmsHubClient> hubContext) : ControllerBase
{
    [HttpPost("{id}/approve")]
    public async Task<IActionResult> Approve(string id,
CancellationToken ct)
    {
        // Your existing approval logic ...

        // After the database commit succeeds, broadcast to all
        connected Angular clients
    }
}
```

```

        await
hubContext.Clients.All.ReceiveEnrollmentStatusUpdated(id,
"Approved");
        return NoContent();
    }
}

```

Design Note: We choose `hubContext.Clients.All` here rather than narrow client groups. For enrollment changes that must alter public metrics instantly across different training centers, a global broadcast is the correct choice.

Step 3: Install the Client Package (Angular Client)

Open a terminal inside your Angular project folder and install the official Microsoft SignalR package:

```
npm install @microsoft/signalr
```

[!NOTE]

Step 4: Configure the Local Development Server Proxy

To prevent cross-origin resource sharing errors or broken path exceptions when executing local connections, you must map local routing intercepts straight toward your .NET Kestrel engine port.

Create a configuration file named `proxy.conf.json` directly inside your project root:

```

{
  "/api": {
    "target": "http://localhost:5000",
    "secure": false,
    "changeOrigin": true
  },
  "/hubs": {
    "target": "http://localhost:5000",
    "secure": false,
    "ws": true
  }
}

```

Note: Enforcing `"ws": true` on the hubs entry is a critical configuration step. It signals the proxy to upgrade the connection handshake to the WebSocket protocol. Without it, the handshake fails and drops down to slow long-polling cycles.

Wire the proxy into your `angular.json` workspace file under the serving choices block:

```
"serve": {
  "options": {
    "proxyConfig": "proxy.conf.json"
  }
}
```

Enforcement Check: Restart your frontend **ng serve** server process after saving this file, as proxy configurations are parsed exclusively during local environment startup loops.

[NOTE]

Step 5: Build the Live Sync Client Service

Generate a message transport manager class via the Angular CLI:

```
ng generate service services/live-sync
```

Open **src/app/services/live-sync.service.ts** and set up the connection state listeners:

```
import { inject, PLATFORM_ID, signal } from "@angular/core";
import { isPlatformBrowser } from "@angular/common";
import { HubConnection, HubConnectionBuilder } from
"@microsoft/signalr";
import { Subject } from "rxjs";

public interface EnrollmentStatusEvent {
  id: string;
  status: 'Pending' | 'Approved' | 'Rejected';
}

@Service()
public class LiveSyncService {
  private platformId = inject(PLATFORM_ID);
  private connection: HubConnection | null = null;
  private eventsSubject = new Subject<EnrollmentStatusEvent>();

  // Expose events as an observable stream for state store
  subscriptions
  events$ = this.eventsSubject.asObservable();

  connectionState = signal<'connected' | 'reconnecting' |
'disconnected'>('disconnected');

  connect() {
    // Guard against duplicate connection setups
    if (this.connection) return;

    // WebSockets are a browser feature. Skip execution if
```

```

evaluated inside a Node.js server loop
    if (!isPlatformBrowser(this.platformId)) return;

    this.connection = new HubConnectionBuilder()
        .withUrl('/hubs/tms')
        .withAutomaticReconnect([0, 2000, 10000, 30000])
        .build();

    // The mapping string matches the exact .NET method contract
    // declared inside ITmsHubClient
    this.connection.on(
        'ReceiveEnrollmentStatusUpdated',
        (enrollmentId: string, status: 'Pending' | 'Approved' |
        'Rejected') => {
            this.eventsSubject.next({ id: enrollmentId, status });
        }
    );

    this.connection.onreconnecting(() =>
    this.connectionState.set('reconnecting'));
    this.connection.onreconnected(() =>
    this.connectionState.set('connected'));
    this.connection.onclose(() =>
    this.connectionState.set('disconnected'));

    this.connection
        .start()
        .then(() => this.connectionState.set('connected'))
        .catch(err => console.error('SignalR connection error:',
err));
    }
}

```

[!NOTE]

Step 6: Bridge Live Events into the SignalStore Engine

Open `src/app/store/enrollment.store.ts`. Isolate transportation from mutations by binding your outbound proxy stream directly inside the `withMethods` block:

```

import { inject } from '@angular/core';
import { signalStore, withMethods, patchState } from
'@ngrx/signals';
import { withEntities, updateEntity } from
'@ngrx/signals/entities';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { pipe, switchMap, tap } from 'rxjs';
import { EnrollmentService } from '../services/enrollment.service';
import { LiveSyncService } from '../services/live-sync.service';
import { Enrollment } from '../models/enrollment.model';

```

```

export const EnrollmentStore = signalStore(
  { providedIn: 'root' },
  withEntities<Enrollment>(),
  withMethods((store, api = inject(EnrollmentService), sync =
    inject(LiveSyncService)) => ({
    // Listens to SignalR live sync stream and updates store state
    automatically
    listenForLiveUpdates: rxMethod<void>(
      pipe(
        tap(() => sync.connect()),
        switchMap(() => sync.events\$,),
        tap(event => {
          patchState(
            store,
            updateEntity({ id: event.id, changes: { status:
              event.status } })
          );
        })
      )
    )
    // ... your existing loadEnrollments() and approveEnrollment()
    methods
  })))
);

```

[!NOTE]

Step 7: Activate the Live Sync Listener on App Startup

Open your root entry file `src/app/app.component.ts` (or `instructor-dashboard.component.ts`) and trigger the state listener loop within your initialization hook:

```

import { Component, OnInit, inject } from '@angular/core';
import { EnrollmentStore } from './store/enrollment.store';

export class AppComponent implements OnInit {
  private store = inject(EnrollmentStore);

  ngOnInit() {
    this.store.loadEnrollments();
    this.store.listenForLiveUpdates();
  }
}

```

Exercise 5 Verification and Real-Time Sync Testing

Follow these verification steps in order to confirm your socket-sync infrastructure:

1. **Initialize Your Services:** Spin up your backend environment (`dotnet run --project TmsApi.Api`) alongside your client dashboard (`ng serve`).

2. **Launch Split-Screen Views:** Open two separate browser windows side by side. Navigate Tab 1 to <http://localhost:4200/enrollments> and Tab 2 straight to <http://localhost:4200/dashboard>.
3. **Trigger State Mutations:** In Tab 1, select a "Pending" record from your data grid and click **Approve**.
4. **Observe Real-Time Updates:** Watch Tab 2 closely. The pending metric total shown on the dashboard drops instantly without requiring a page refresh or manual API polling loops, proving your SignalR connection context is updating the unified state store across windows.